ОТЧЁТ

по лабораторной работе №7

«Принципы SOLID и рефакторинг архитектуры Python-приложения»

Вариант 20. Интернет-провайдер

1. Цель работы

Научиться находить нарушения SOLID и перестраивать приложение так, чтобы подключение абонента, расчёт оплаты, хранение и отчёт были разделены, а новая тарифная стратегия добавлялась без правки уже написанного сервиса.

2. Постановка индивидуального задания

Вариант 20. Система управления абонентами и тарифами. Операции: подключение клиента, изменение тарифа, блокировка и расчёт оплаты. Нужны различные тарифные стратегии.

Сначала показана исходная «монолитная» версия с нарушениями, затем рабочая архитектура: репозиторий, стратегии, узкие уведомления и сервис. Поведение покрыто автоматическими тестами.

3. Краткие теоретические сведения

S — у класса одна причина для изменения. O — класс открыт для расширения и закрыт для правки: новая стратегия добавляется новым классом. L — наследник можно подставить вместо базового типа без поломки ожиданий. I — интерфейс не должен заставлять реализовывать лишние методы. D — сервис зависит от абстракции, а конкретные объекты передаются снаружи.

В исходном ProviderMonolith расчёт, запись файла и печать отчёта смешаны, новый тариф требует ветки if, а способ оплаты зашит внутри класса. После рефакторинга эти обязанности разведены.

4. Описание программы

HomeStrategy, CorporateStrategy и MobileStrategy реализуют один и тот же расчёт месячной платы. StrategyRegistry находит стратегию по коду и позволяет зарегистрировать ещё одну. MemorySubscriberRepository хранит абонентов. TextReport только печатает отчёт. ConsoleNotifier разделён на уведомление об оплате и о блокировке.

SubscriberService получает репозиторий, реестр и уведомители в конструкторе. Подключение создаёт абонента и считает сумму. Смена тарифа запрещена заблокированному абоненту. Блокировка обнуляет дальнейшую оплату. Тесты проверяют домашний тариф, корпоративную скидку, смену тарифа, нулевую оплату после блокировки, неизвестный код и повторное подключение.

5. Исходный код программы

Ниже приведён исходный код разработанных файлов.

Листинг — main.py

```
"""
Лабораторная работа №7. Вариант 20.
Интернет-провайдер после рефакторинга по SOLID.
"""

import sys

from billing import CorporateStrategy, HomeStrategy, MobileStrategy, StrategyRegistry
from notifier import ConsoleNotifier
from reports import TextReport
from repository import MemorySubscriberRepository
from service import SubscriberService

if sys.platform == "win32":
    sys.stdout.reconfigure(encoding="utf-8")
    sys.stderr.reconfigure(encoding="utf-8")
    if not sys.stdin.isatty():
        sys.stdin.reconfigure(encoding="utf-8")



def read_line(prompt):
    if sys.stdin.isatty():
        return input(prompt).strip()
    raw = sys.stdin.readline()
    if raw == "":
        raise EOFError
    text = raw.rstrip("\r\n")
    print(f"{prompt}{text}")
    return text.strip()



def read_positive_int(prompt):
    while True:
        text = read_line(prompt)
        if text.isdigit() and int(text) > 0:
            return int(text)
        print("Ошибка: введите целое положительное число.")



def build_service():
    repository = MemorySubscriberRepository()
    registry = StrategyRegistry([HomeStrategy(), CorporateStrategy(), MobileStrategy()])
    notifier = ConsoleNotifier()
    service = SubscriberService(repository, registry, notifier, notifier)
    return service, repository



def print_menu():
    print()
    print("===== Провайдер (SOLID) =====")
    print("1. Подключить клиента")
    print("2. Изменить тариф")
```

```
    print("3. Заблокировать")
    print("4. Рассчитать оплату")
    print("5. Отчёт")
    print("0. Выход")
    print("Тарифы: home, corp, mobile")
    print()


def main():
    service, repository = build_service()
    print("Зависимости собраны снаружи: репозиторий, стратегии, уведомления.")
    while True:
        print_menu()
        choice = read_line("Выберите действие: ")
        try:
            if choice == "1":
                name = read_line("Имя клиента: ")
                code = read_line("Код тарифа: ")
                months = read_positive_int("Месяцев: ")
                amount = service.connect(name, code, months)
                print(f"Клиент подключён. Начислено {amount:.0f} руб.")
            elif choice == "2":
                name = read_line("Имя клиента: ")
                code = read_line("Новый код тарифа: ")
                amount = service.change_tariff(name, code)
                print(f"Тариф изменён. Новая сумма {amount:.0f} руб.")
            elif choice == "3":
                name = read_line("Имя клиента: ")
                service.block(name)
            elif choice == "4":
                name = read_line("Имя клиента: ")
                print(f"К оплате: {service.payment(name):.0f} руб.")
            elif choice == "5":
                print(TextReport(repository).render())
            elif choice == "0":
                print("Работа программы завершена.")
                break
            else:
                print("Ошибка: выберите пункт от 0 до 5.")
        except ValueError as exc:
            print(f"Ошибка: {exc}")
        except EOFError:
            break


if __name__ == "__main__":
    main()
```

Листинг — billing.py

```
"""Стратегии расчёта оплаты. Новая стратегия не меняет сервис (OCP)."""

from typing import Protocol


class PaymentStrategy(Protocol):
    code: str

    def monthly_fee(self) -> float:
        ...


class HomeStrategy:
    code = "home"

    def monthly_fee(self) -> float:
        return 500.0


class CorporateStrategy:
    code = "corp"

    def monthly_fee(self) -> float:
```

```
        return 2500.0 * 0.9


class MobileStrategy:
    code = "mobile"

    def monthly_fee(self) -> float:
        return 350.0



class StrategyRegistry:
    def __init__(self, strategies: list[PaymentStrategy]):
        self._items = {item.code: item for item in strategies}

    def get(self, code: str) -> PaymentStrategy:
        if code not in self._items:
            raise ValueError(f"неизвестный тариф: {code}")
        return self._items[code]

    def register(self, strategy: PaymentStrategy) -> None:
        self._items[strategy.code] = strategy
```

Листинг — legacy\monolith.py

```
"""
Первоначальная версия (для анализа нарушений SOLID).
Один класс одновременно хранит данные, считает оплату, печатает отчёт
и сам создаёт файловое хранилище. Новый тариф требует правки if/elif.
"""



class ProviderMonolith:
    def __init__(self):
        self.clients = []
        self.path = "clients.txt"

    def connect_client(self, name, tariff_code, months):
        # Нарушение OCP: новая стратегия — новая ветка в этом методе.
        if tariff_code == "home":
            payment = 500 * months
        elif tariff_code == "corp":
            payment = 2500 * months * 0.9
        elif tariff_code == "mobile":
            payment = 350 * months
        else:
            raise ValueError("неизвестный тариф")
        self.clients.append({"name": name, "tariff": tariff_code, "payment": payment, "blocked": False})
        self.save_and_print_report()
        return payment

    def change_tariff(self, name, tariff_code, months):
        for client in self.clients:
            if client["name"] == name:
                client["tariff"] = tariff_code
                return self.connect_client(name + "_copy", tariff_code, months)
        return None

    def block(self, name):
        for client in self.clients:
            if client["name"] == name:
                client["blocked"] = True

    def save_and_print_report(self):
        # Нарушение SRP: сохранение и отчёт смешаны с подключением.
        lines = []
        for client in self.clients:
            line = f"{client['name']};{client['tariff']};{client['payment']}"
            lines.append(line)
            print("ОТЧЁТ:", line)
        with open(self.path, "w", encoding="utf-8") as file:
            file.write("\n".join(lines))

    def charge_external_api(self, client):
```

```
        # Нарушение DIP: конкретный способ оплаты зашит в класс.
        print("Списание через конкретный банк:", client["name"])
```

Листинг — models.py

```
"""Модели абонента после рефакторинга."""

from dataclasses import dataclass



@dataclass
class Subscriber:
    name: str
    tariff_code: str
    blocked: bool = False
    months: int = 1
```

Листинг — notifier.py

```
"""Узкие интерфейсы вместо одного «толстого» (ISP)."""

from typing import Protocol



class BlockNotifier(Protocol):
    def notify_blocked(self, name: str) -> None:
        ...



class PaymentNotifier(Protocol):
    def notify_payment(self, name: str, amount: float) -> None:
        ...



class ConsoleNotifier:
    def notify_blocked(self, name: str) -> None:
        print(f"Уведомление: абонент {name} заблокирован")

    def notify_payment(self, name: str, amount: float) -> None:
        print(f"Уведомление: с абонента {name} начислено {amount:.0f} руб.")
```

Листинг — reports.py

```
"""Формирование отчёта вынесено из сервиса подключения (SRP)."""

from repository import SubscriberRepository



class TextReport:
    def __init__(self, repository: SubscriberRepository):
        self.repository = repository

    def render(self) -> str:
        lines = ["Абонент | Тариф | Месяцев | Блокировка"]
        for item in self.repository.all():
            blocked = "да" if item.blocked else "нет"
            lines.append(f"{item.name} | {item.tariff_code} | {item.months} | {blocked}")
        return "\n".join(lines)
```

Листинг — repository.py

```
"""Хранилище абонентов. Сервис зависит от абстракции, а не от файла."""

from typing import Protocol

from models import Subscriber



class SubscriberRepository(Protocol):
    def add(self, subscriber: Subscriber) -> None:
        ...
```

```
    def all(self) -> list[Subscriber]:
        ...

    def find(self, name: str) -> Subscriber | None:
        ...


class MemorySubscriberRepository:
    def __init__(self):
        self._items: list[Subscriber] = []

    def add(self, subscriber: Subscriber) -> None:
        self._items.append(subscriber)

    def all(self) -> list[Subscriber]:
        return list(self._items)

    def find(self, name: str) -> Subscriber | None:
        for item in self._items:
            if item.name.lower() == name.lower():
                return item
        return None
```

Листинг — service.py

```
"""Сервис подключения. Зависимости передаются снаружи (DIP)."""

from billing import StrategyRegistry
from models import Subscriber
from notifier import BlockNotifier, PaymentNotifier
from repository import SubscriberRepository


class SubscriberService:
    def __init__(
        self,
        repository: SubscriberRepository,
        strategies: StrategyRegistry,
        payment_notifier: PaymentNotifier,
        block_notifier: BlockNotifier,
    ):
        self.repository = repository
        self.strategies = strategies
        self.payment_notifier = payment_notifier
        self.block_notifier = block_notifier

    def connect(self, name: str, tariff_code: str, months: int) -> float:
        if months <= 0:
            raise ValueError("число месяцев должно быть больше нуля")
        if self.repository.find(name) is not None:
            raise ValueError("абонент уже подключён")
        strategy = self.strategies.get(tariff_code)
        subscriber = Subscriber(name, tariff_code, False, months)
        self.repository.add(subscriber)
        amount = strategy.monthly_fee() * months
        self.payment_notifier.notify_payment(name, amount)
        return amount

    def change_tariff(self, name: str, tariff_code: str) -> float:
        subscriber = self.repository.find(name)
        if subscriber is None:
            raise ValueError("абонент не найден")
        if subscriber.blocked:
            raise ValueError("нельзя сменить тариф заблокированному абоненту")
        strategy = self.strategies.get(tariff_code)
        subscriber.tariff_code = tariff_code
        amount = strategy.monthly_fee() * subscriber.months
        self.payment_notifier.notify_payment(name, amount)
        return amount

    def block(self, name: str) -> None:
        subscriber = self.repository.find(name)
        if subscriber is None:
```

```
            raise ValueError("абонент не найден")
        subscriber.blocked = True
        self.block_notifier.notify_blocked(name)

    def payment(self, name: str) -> float:
        subscriber = self.repository.find(name)
        if subscriber is None:
            raise ValueError("абонент не найден")
        if subscriber.blocked:
            return 0.0
        strategy = self.strategies.get(subscriber.tariff_code)
        return strategy.monthly_fee() * subscriber.months
```

Листинг — tests\test_service.py

```
"""Тесты бизнес-логики после рефакторинга."""

import os
import sys

sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from billing import CorporateStrategy, HomeStrategy, MobileStrategy, StrategyRegistry
from notifier import ConsoleNotifier
from repository import MemorySubscriberRepository
from service import SubscriberService



class SilentNotifier:
    def notify_blocked(self, name: str) -> None:
        pass

    def notify_payment(self, name: str, amount: float) -> None:
        pass



def make_service():
    repository = MemorySubscriberRepository()
    registry = StrategyRegistry([HomeStrategy(), CorporateStrategy(), MobileStrategy()])
    notifier = SilentNotifier()
    return SubscriberService(repository, registry, notifier, notifier), repository



def test_connect_home():
    service, _ = make_service()
    assert service.connect("Иванов", "home", 2) == 1000



def test_corporate_discount():
    service, _ = make_service()
    assert service.connect("Офис", "corp", 1) == 2250



def test_change_tariff():
    service, _ = make_service()
    service.connect("Петрова", "mobile", 2)
    assert service.change_tariff("Петрова", "home") == 1000



def test_blocked_payment_is_zero():
    service, _ = make_service()
    service.connect("Сидоров", "home", 3)
    service.block("Сидоров")
    assert service.payment("Сидоров") == 0



def test_unknown_tariff():
    service, _ = make_service()
    try:
        service.connect("Орлов", "vip", 1)
        assert False
```

```
    except ValueError:
        assert True


def test_duplicate_rejected():
    service, _ = make_service()
    service.connect("Иванов", "home", 1)
    try:
        service.connect("Иванов", "mobile", 1)
        assert False
    except ValueError:
        assert True
```

6. Примеры работы программы

Снимки показывают подключение клиентов, смену тарифа, блокировку и прогон тестов бизнес-логики.

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python7> python main.py

Зависимости собраны снаружи: репозиторий, стратегии, уведомления.

===== Провайдер (SOLID) =====
1. Подключить клиента
2. Изменить тариф
3. Заблокировать
4. Рассчитать оплату
5. Отчёт
0. Выход
Тарифы: home, corp, mobile

Выберите действие: 1
Имя клиента: Иванов
Код тарифа: home
Месяцев: 2
Уведомление: с абонента Иванов начислено 1000 руб.
Клиент подключён. Начислено 1000 руб.

===== Провайдер (SOLID) =====
1. Подключить клиента
2. Изменить тариф
3. Заблокировать
4. Рассчитать оплату
5. Отчёт
0. Выход
Тарифы: home, corp, mobile

Выберите действие: 1
Имя клиента: Петрова
Код тарифа: corp
Месяцев: 1
Уведомление: с абонента Петрова начислено 2250 руб.
Клиент подключён. Начислено 2250 руб.

===== Провайдер (SOLID) =====
1. Подключить клиента
2. Изменить тариф
3. Заблокировать
4. Рассчитать оплату
5. Отчёт
0. Выход
Тарифы: home, corp, mobile

Выберите действие: 5
Абонент | Тариф | Месяцев | Блокировка
Иванов | home | 2 | нет
Петрова | corp | 1 | нет
```

Рисунок 1 – Подключение домашнего и корпоративного тарифа и отчёт

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python7> python main.py

Зависимости собраны снаружи: репозиторий, стратегии, уведомления.

===== Провайдер (SOLID) =====
1. Подключить клиента
2. Изменить тариф
3. Заблокировать
4. Рассчитать оплату
5. Отчёт
0. Выход
Тарифы: home, corp, mobile

Выберите действие: 1
Имя клиента: Иванов
Код тарифа: home
Месяцев: 2
Уведомление: с абонента Иванов начислено 1000 руб.
Клиент подключён. Начислено 1000 руб.

===== Провайдер (SOLID) =====
1. Подключить клиента
2. Изменить тариф
3. Заблокировать
4. Рассчитать оплату
5. Отчёт
0. Выход
Тарифы: home, corp, mobile

Выберите действие: 2
Имя клиента: Иванов
Новый код тарифа: mobile
Уведомление: с абонента Иванов начислено 700 руб.
Тариф изменён. Новая сумма 700 руб.

===== Провайдер (SOLID) =====
1. Подключить клиента
2. Изменить тариф
3. Заблокировать
4. Рассчитать оплату
5. Отчёт
0. Выход
Тарифы: home, corp, mobile

Выберите действие: 4
Имя клиента: Иванов
К оплате: 700 руб.

===== Провайдер (SOLID) =====
```

Рисунок 2 – Смена тарифа, расчёт оплаты и блокировка абонента

```
C:\Users\Shmurdyakich\Desktop\pythonALL\python7> python main.py

Зависимости собраны снаружи: репозиторий, стратегии, уведомления.

===== Провайдер (SOLID) =====
1. Подключить клиента
2. Изменить тариф
3. Заблокировать
4. Рассчитать оплату
5. Отчёт
0. Выход
Тарифы: home, corp, mobile

Выберите действие: 1
Имя клиента: Орлов
Код тарифа: vip
Месяцев: 1
Ошибка: неизвестный тариф: vip

===== Провайдер (SOLID) =====
1. Подключить клиента
2. Изменить тариф
3. Заблокировать
4. Рассчитать оплату
5. Отчёт
0. Выход
Тарифы: home, corp, mobile

Выберите действие: 1
Имя клиента: Иванов
Код тарифа: home
Месяцев: 1
Уведомление: с абонента Иванов начислено 500 руб.
Клиент подключён. Начислено 500 руб.

===== Провайдер (SOLID) =====
1. Подключить клиента
2. Изменить тариф
3. Заблокировать
4. Рассчитать оплату
5. Отчёт
0. Выход
Тарифы: home, corp, mobile

Выберите действие: 0
Работа программы завершена.
```

Рисунок 3 – Отказ неизвестного тарифа и повторного подключения

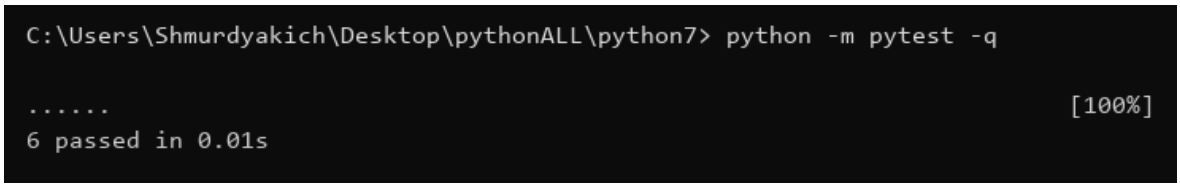


Рисунок 4 – Автоматические тесты сервиса

7. Ответы на контрольные вопросы

1. В чём смысл SRP?

Класс меняют по одной причине. Отчёт, хранение и подключение абонента в итоговой версии живут в разных модулях.

2. Как здесь соблюдён OCP?

Новая тарифная стратегия — это новый класс, который регистрируется в StrategyRegistry. Сервис подключения при этом не переписывается.

3. Что нарушал исходный if по коду тарифа?

Каждый новый тариф требовал правки того же метода. Это закрытый для расширения код.

4. Как проявляется DIP?

Сервис не создаёт файл и не выбирает конкретный класс хранилища сам. Репозиторий и стратегии передаются в конструктор.

5. Зачем два узких интерфейса уведомлений?

Это ISP. Коду блокировки не нужен метод оплаты, а коду оплаты не нужен метод блокировки.

6. Почему тесты бьют по сервису, а не по меню?

Меню только вводит данные. Правила подключения, смены тарифа и блокировки проверяются напрямую и не зависят от консоли.

8. Вывод

В лабораторной работе №7 монолитный учёт абонентов варианта 20 перестроен по SOLID. Оплата задаётся стратегиями, данные хранит репозиторий, отчёт и уведомления вынесены отдельно, а сервис получает зависимости снаружи. Тесты подтверждают подключение, смену тарифа, расчёт и блокировку.